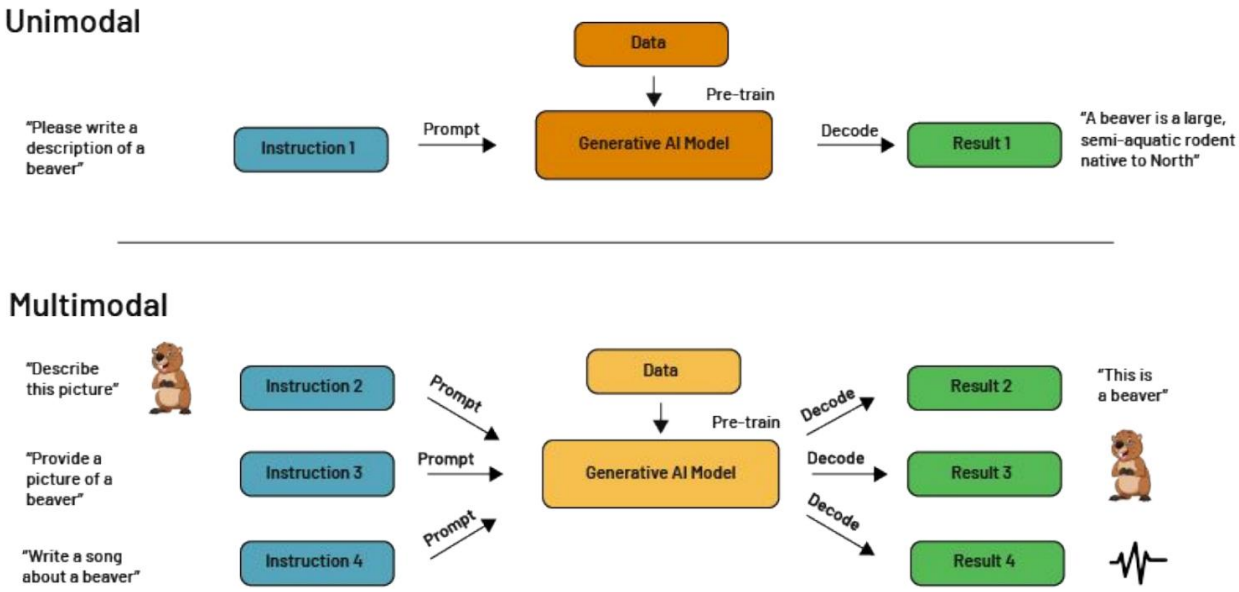
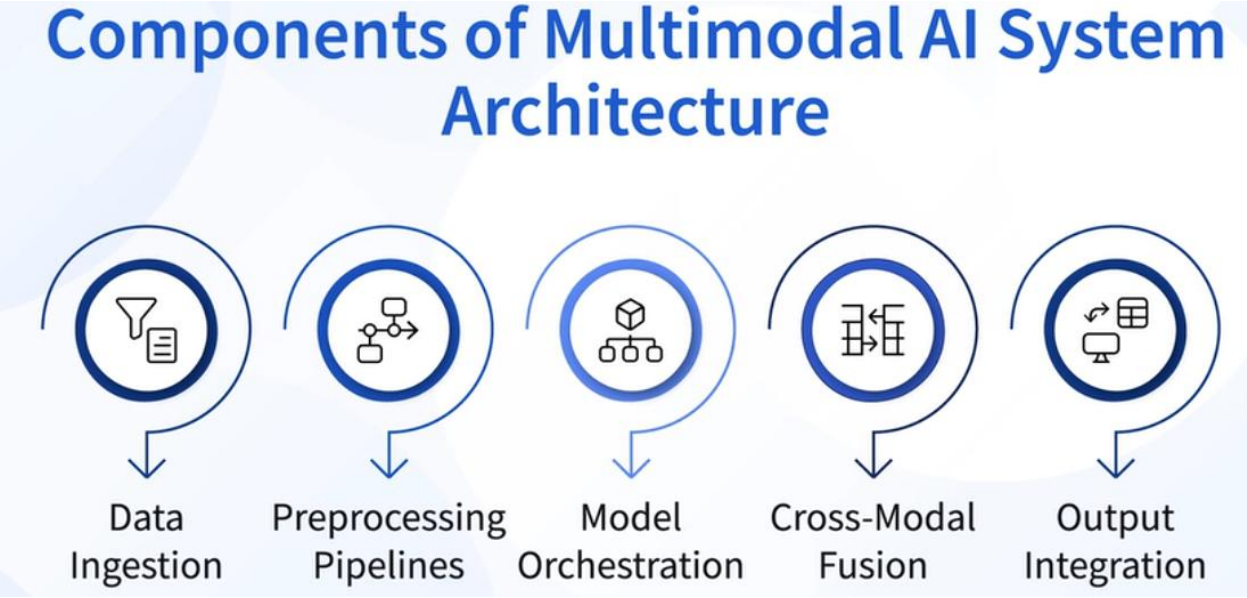
Multimodal Models

Multimodal models can process multiple types of data simultaneously, such as images, text, and sound. This means they can understand the complex relationship between a picture and the words used to describe it, or between a sound and the emotion that it conveys. A multimodal model could take an image of a beaver as a prompt and generate a text result that says, "This is a beaver". It could also take a text prompt like "Provide a picture of a beaver" and generate an image. It can even take an instruction to "Write a song about a beaver" and generate an audio file as a result.





Data Ingestion Layer

- Handles accepting and routing different data types from various sources, ensuring data arrives in the correct format and timing.
- Acts as a traffic controller managing data flow to appropriate processing pipelines.

Data Preprocessing Pipeline

- Cleans, normalizes, and standardizes raw multi-modal data for AI model consumption.
- Uses tools like OpenCV for images, Librosa for audio, and Hugging Face tokenizers for text to ensure consistent input quality.

1. Tabular Data (Demographics, Vitals, Lab Results)

Raw Format: A mix of continuous numbers (e.g., Blood Pressure: 120, Age: 45) and categorical values (e.g., Gender: "Male", Smoker: "Yes").

How it is Converted:

Categorical text values are converted to numbers using One-Hot Encoding or learned Embedding Layers.

Continuous numbers are normalized or standardized (e.g., scaled between 0 and 1).

These numbers are passed through a simple Fully Connected (Dense) Layer to map them to the desired size.

The Resulting Feature Form: A 1D Dense Vector of size [Batch_Size, Tabular_Features] (e.g., a flat array of 32 or 64 numbers representing the patient's physical profile).

2. Clinical Notes (Free-Text)

Raw Format: Unstructured text strings written by doctors (e.g., "Patient complains of chronic migraines and localized numbness...").

How it is Converted:

The text is tokenized into sub-word units.

These tokens are fed into a Pre-trained Text Transformer or Large Language Model optimized for medicine (like ClinicalBERT or BioBERT).

The model processes the sentence context and extracts the final hidden state or the specialized token representation.

The Resulting Feature Form: A 1D Semantic Vector of size [Batch_Size, Text_Hidden_Dim] (commonly a flat vector of 768 or 1024 numbers capturing the clinical meaning of the notes).

3. MRI Scan (3D Medical Image)

Raw Format: A 3D spatial volume of voxel intensities representing anatomical cross-sections.

How it is Converted:

The 3D scan is fed into a 3D Convolutional Neural Network (3D-CNN) or a 3D Vision Transformer (ViT).

The network processes the image layers, compressing the spatial dimensions (width, height, depth) through pooling while extracting increasingly abstract visual patterns (edges, tissue densities, anomalies).

The final convolutional layer is flattened or global-average-pooled.

The Resulting Feature Form: A 1D Visual Feature Vector of size [Batch_Size, Image_Features] (e.g., a vector of 512 or 2048 numbers summarizing the structural characteristics of the scan).

4. Patient Audio (Voice Recording)

Raw Format: A continuous 1D raw waveform (amplitude over time).

How it is Converted:

The audio is typically converted into a 2D visual representation of sound over time, called a Mel-Spectrogram, or processed natively using an audio network.

This representation is passed through an Audio Encoder like Wav2Vec 2.0, an Audio Spectrogram Transformer (AST), or a standard 2D-CNN.

The network extracts acoustic features like pitch, tone, jitter, and speech cadence, pooling them into a single summary.

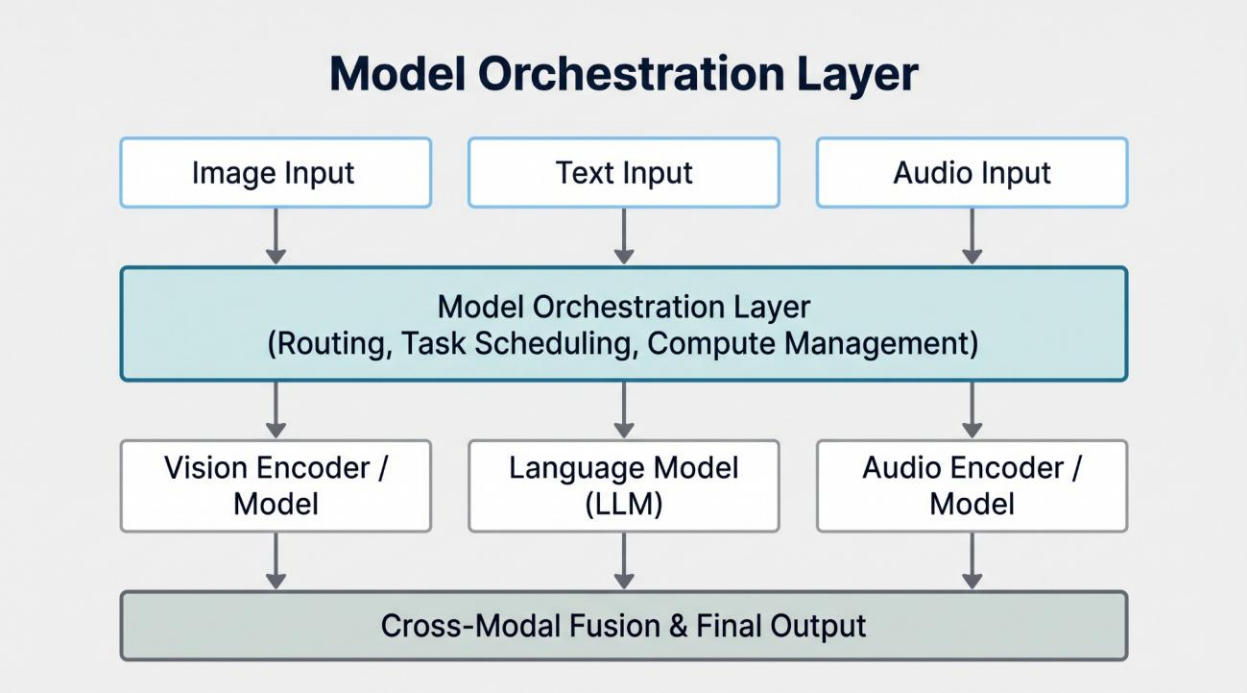
The Resulting Feature Form: A 1D Acoustic Feature Vector of size [Batch_Size, Audio_Features] (e.g., a vector of 256 or 512 numbers capturing the vocal markers of the patient).

Model Orchestration Services

- Coordinates multiple AI models processing different data modalities in parallel.
- Manages resource allocation, handles failures, and ensures consistent response times using workflow managers or orchestration platforms.

Cross-Model Fusion and Integration

- Combines insights from different AI models into unified, actionable results.
- Integrates outputs like text sentiment, image analysis, and audio emotion recognition for comprehensive assessments.



Output Integration and API Layer

- Presents unified results to end-users and downstream systems through consistent interfaces.
- Handles authentication, rate-limiting, and formats responses for various applications.

Understanding the core components

At its heart, a multimodal AI system is an organized structure of specialized software components that work together. Each part has a specific job, from receiving initial data to producing a final, useful output. Think of it like a team of experts, where each one specializes in a different sense—one for sight, one for language—and they collaborate to form a complete understanding. To build such a system, developers rely on several key components that form the foundation of its architecture.

Receiving different types of data

The first step for any multimodal system is to take in information from the world. These inputs are called **data streams**, and they can come in many forms. For an AI to understand an image, it needs **an image input stream**. This is the pathway through which visual information, like a photograph or a frame from a video, enters the system. Similarly, to understand a question or a command, it needs a **textual input stream**. This is where written words are fed into the system. Other systems might also include **audio streams** for spoken language or even **numerical data streams** from sensors. Each stream is a dedicated channel for a specific type of data, ensuring that the information is received correctly before processing begins.

Translating data for the AI

Computers do not see images or read text the way humans do. They understand numbers. Therefore, the next crucial step is to translate, or encode, the input data into a numerical format that the AI can work with. This job is handled by **encoders**. An image encoder is typically a type of model known as a **Convolutional Neural Network (CNN)**. A CNN is designed to recognize patterns in visual data. It breaks an image down into its basic elements, like lines, shapes, colors, and textures, and converts these features into a set of numbers. A text encoder, on the other hand, is often a transformer-based model. These models are experts at understanding language. They analyze the relationships between words in a sentence to capture meaning, context, and grammar, turning the text into its own unique set of numbers, often called a vector.

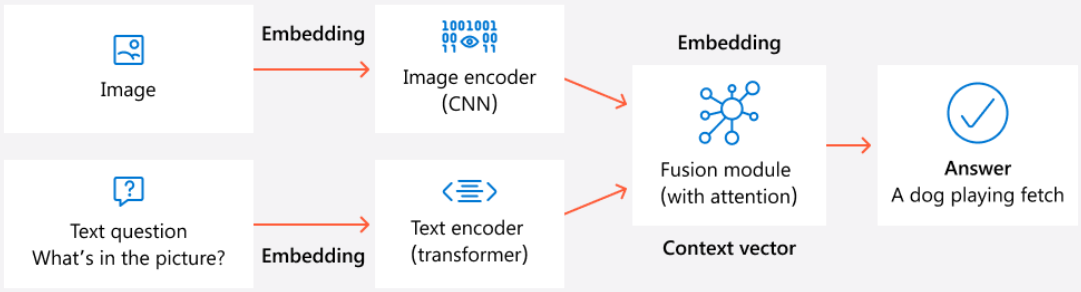
Combining information for a unified view

The **fusion** module is a critical component that combines encoded image and text data. Fusion integrates information from different modalities to form a unified understanding. Simple methods like concatenation stack the data, while advanced attention mechanisms allow the system to weigh information's importance. For example, when asked about a car's color, **attention** helps the model focus on the car in the image and ignore the background. This selective focus is key to generating accurate and relevant answers.

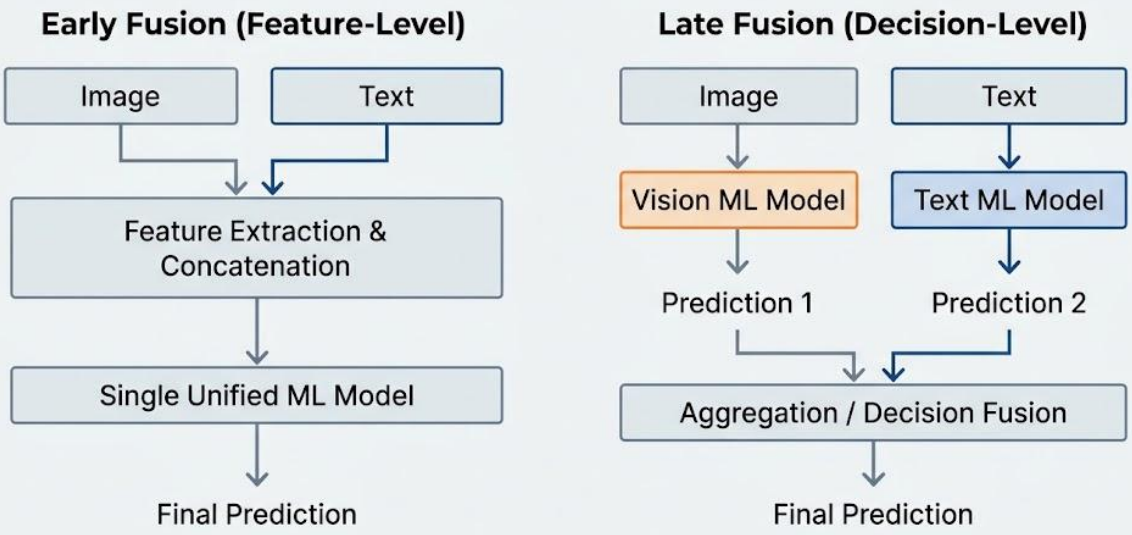
Generating a human-friendly answer

After the fusion module has created a combined representation of the data, this information is passed to a **decoder**. The decoder's job is to take the fused numerical data and translate it back into a format that humans can understand. In a system designed to answer questions, the decoder would generate a text-based answer. For example, it might take the fused data representing "car" and "red" and produce the sentence, "The car is red." In other applications, a decoder might generate a caption for an image, translate text into another language, or even create a new image based on a text description. The decoder is the final step in the process, turning the AI’s internal understanding into a useful output.

Visual Question Answering (VQA) Architecture



Early vs. Late Fusion



When to use?

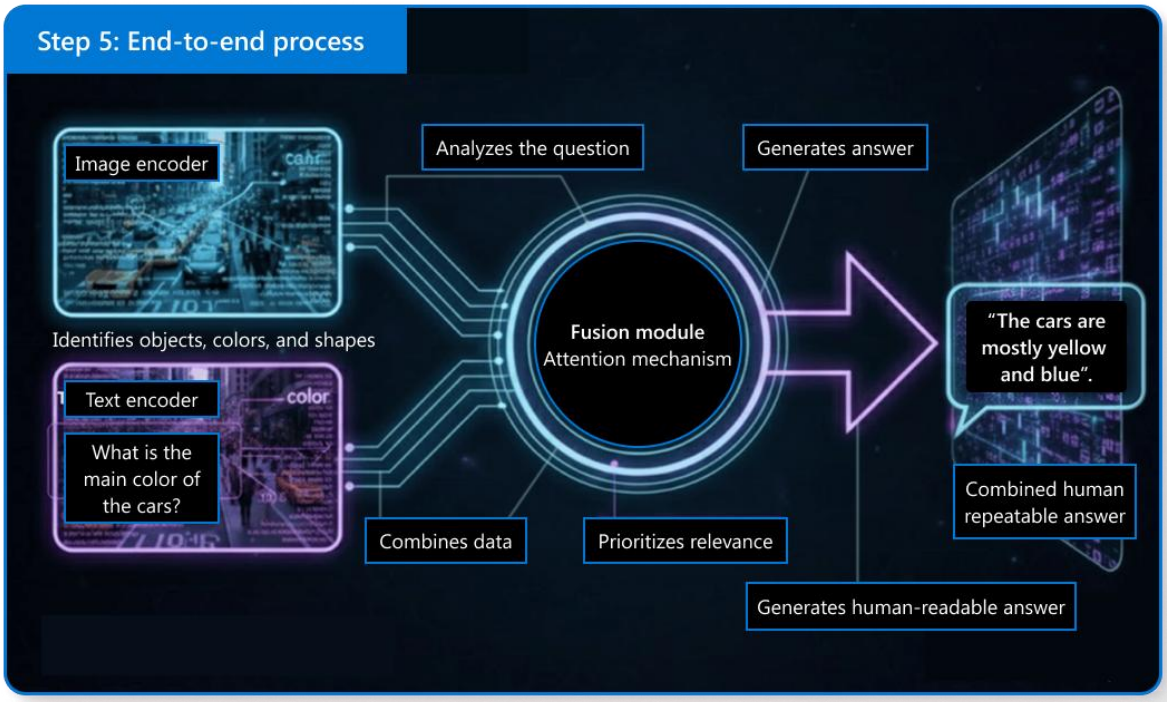
As a golden rule: **Early fusion** is preferred when modalities are highly dependent on each other to make sense, while **late fusion** is preferred when each modality can independently provide a reliable answer.

Early fusion techniques are Raw Data Concatenation, Feature Vector Concatenation etc. Early fusion is ideal for tasks requiring **high precision, spatial alignment, or real-time tracking**, where one modality directly explains or modifies the other.

- Autonomous Driving & Robotics (3D Object Detection)
- Audio-Visual Speech Recognition (Lip Reading in Noise)
- Multimodal Sentiment Analysis (Video + Text + Audio)
- Medical Imaging Cross-Registration (PET + CT Scans)

Late fusion techniques include Majority Voting, Weighted Averaging, Maximum / Minimum Rule, Stacked Generalization, etc. Late fusion is ideal for tasks involving **complex, asynchronous data, high noise, or modular engineering constraints**, where each stream can form its own logical conclusion.

- Multimedia Video Retrieval & Event Detection
- Biometric Authentication (Face + Fingerprint + Voice)
- Medical Diagnosis from Clinical Notes + Lab Reports
- Systems with Missing Modalities (Robustness-Critical Tasks)


Step 5: End-to-end process
Image encoder
Identifies objects, colors, and shapes
Text encoder
What is the main color of the cars?
Analyzes the question
Combines data
Fusion module
Attention mechanism
Prioritizes relevance
Generates answer
Generates human-readable answer
"The cars are mostly yellow and blue".
Combined human repeatable answer